

Introduction to lunar photogrammetry and optical libration determination

Spring 2026 AE-451 Project 1

(Notebook originally created by Fabrizio Pinto, <https://osf.io/tehp2/>)

Created: 7 May 2020 | Jupyter/Wolfram Mathematica: 10 April 2022

Last update on: 28 April 2026

Student version adapted: 12 May 2026

Student: Kaoutar Ammara

This is ver. 1.1 of this document. Any needed updates will appear as separate, follow-up versions.

"Introduction to lunar photogrammetry and optical libration determination" by Fabrizio Pinto is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.

<https://creativecommons.org/licenses/by/4.0/>

Attribution — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

Abstract

Here you shall practice photogrammetry applied to selenodesy, that is, the measurement of lunar features and the determination of their selenographic coordinates from an image of the Moon, taken, in this case, with a telescope. Furthermore, you shall use information derived from your own measurements to calculate the optical lunar libration in latitude and longitude at the date of observation. Finally, you shall compare your selenographic coordinate results to values given in lunar atlases and your libration values to those provided in published lunar ephemerides.

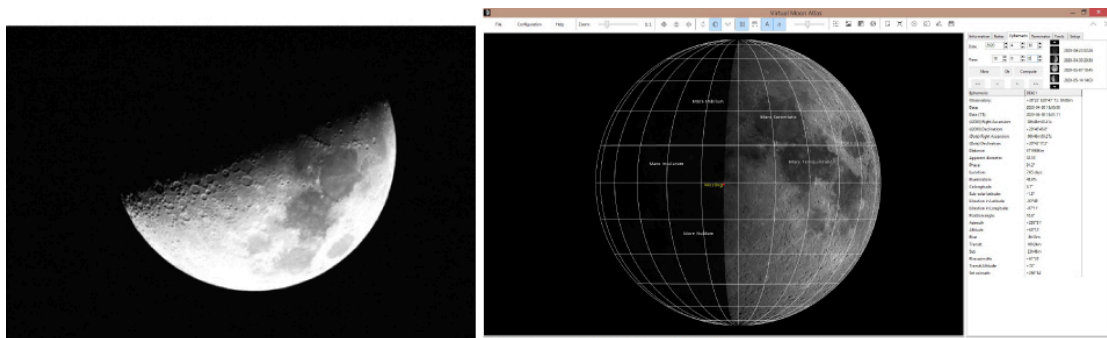


Fig. 1

Fig. 1: (*Left*) The Moon photographed on 30 April 2020 (PHOTO: F. Pinto); (*Right*) The Virtual Moon Atlas main screen.

1. Introduction

Note:

After executing any item, check off the box by typing the word "checked" (without double quotes) after the doubled quotes. The language to check off a box is here at an invisible comments line (open the cell to view it):

Example ☐ ☒

Your **goals** are:

Goal 1. To connect an actual image of the Moon to the information available from an atlas by the determination of the *plate constants* (Fig. 1).

Goal 2. To obtain the scale of the image (km/pixel).

Goal 3. To estimate the libration in longitude and latitude.

Goal 4. To estimate the errors in your results by considering the residuals of the best-fit procedures.

NEEDED TOOLS:

☐ **1. Download the data files provided from my OSF area** (<https://osf.io/tehp2/>) at

Urla Iskelesi Station

Urla Iskelesi Station: Selenography

Urla Iskelesi Station Selenography: Meade ETX-90

Files

2023-04-13

ScienceFrames

☐ At this point, reproduce the entire folder/subfolder structure you find at my cloud folder for the day of 2023-04-13.

☐ Store the FITS image found at /ScienceFrames into folder:

C:/AE-451/DataFiles/2023-04-13/ScienceFrames

This is the image to be reduced (the plate to be solved), provided in FITS format.

☐ Also, download the test data file

fabrizio-pinto-everybody-5980-04302020-2 to your folder

C:/AE-451/DataFiles/2023-04-13/Measurements .

Please note that this file does *not* refer to the image you downloaded and it is an unrelated example only.

☐ **2. ** Download/update AstrolmageJ.****

Download/update to the latest version by following the link available at the FOSS Freeplane mindmap.

☐ **3. Download/update the Virtual Moon Atlas (VMA).**

The VMA can also simulate the illumination conditions at the epoch the image was acquired.

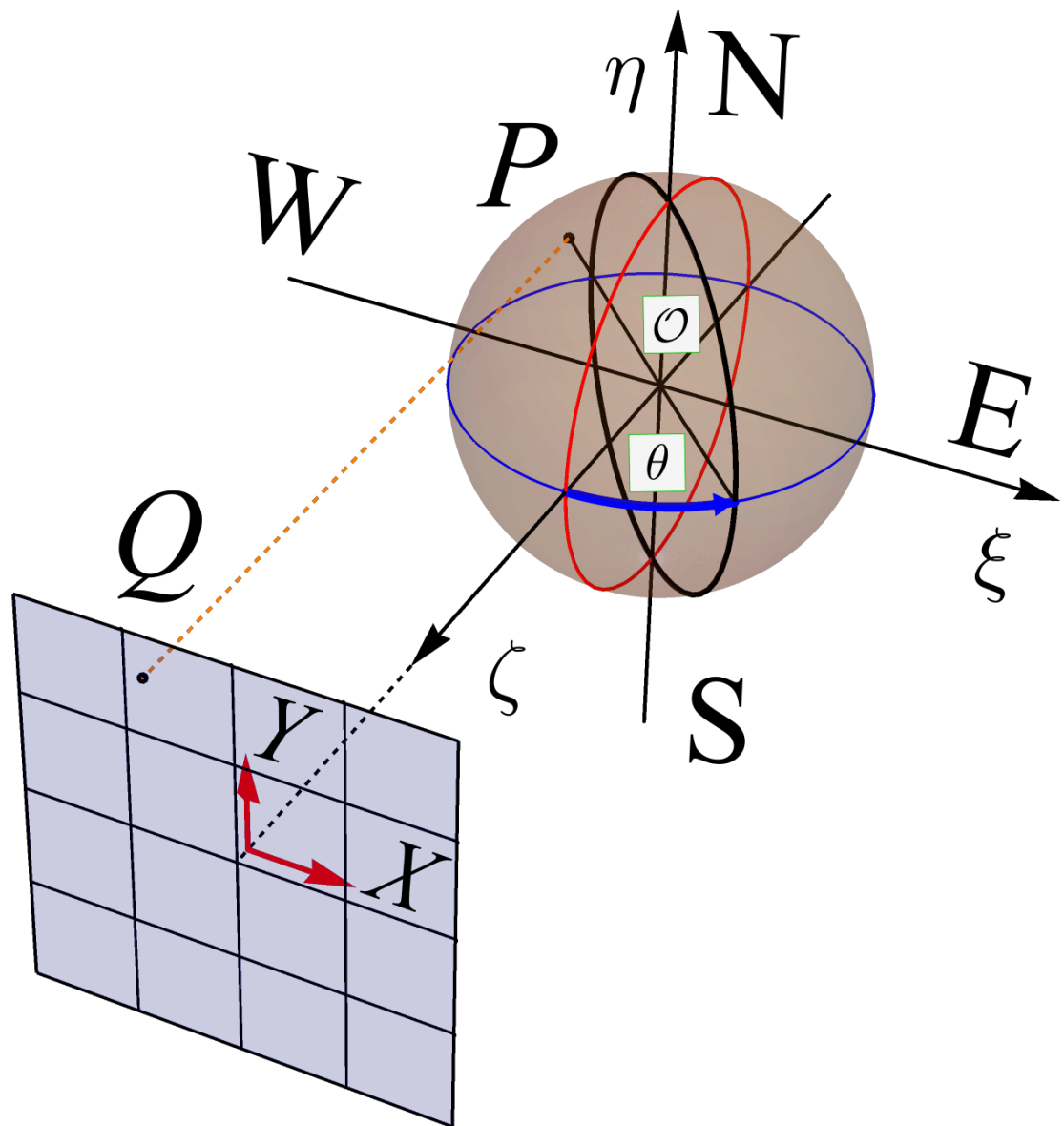
Download/update to the latest version of the VMA by following the link available at the FOSS Freeplane mindmap.

☐ **4. Be ready to launch Notepad or Notepad++.**

This application will be used to record your observations and the feature coordinates in pure ASCII characters. Feel free to use **Notepad++** if you find it helpful.

2. Selenographic coordinates

As also on Earth, the position of any point $\mathbf{P}$ on the Moon is identified by means of its selenographic longitude λ and latitude β . The definitions needed to assign such coordinates are described in the References and are discussed in class. Here, for simplicity, we shall neglect parallactic effects due to the finite distance of the Moon from the Earth and we shall assume the image as representing the Moon from an infinite distance. By means of AstrolmageJ, the pixel coordinates of every point (x, y) on the CCD sensor must be connected to the actual selenographic coordinates (λ, β) .



From the definitions above, the Cartesian components of point $\mathbf{P}$ in the $(\hat{\xi}, \hat{\eta}, \hat{\zeta})$ reference frame are:

$$\xi_P = \cos \beta_P \sin \lambda_P, \quad (1)$$

$$\eta_P = \sin \beta_P, \quad (1)$$

$$\zeta_P = \cos \beta_P \cos \lambda_P, \quad (2)$$

where, if the Moon is assumed to be spherical, $\xi_P^2 + \eta_P^2 + \zeta_P^2 = 1$. Therefore, the relationships, assumed linear, connecting the plate coordinates (x_P, y_P) to the coordinates (ξ_P, η_P, ζ_P) are:

$$x_P = A\xi_P + B\eta_P + C\zeta_P + D, \quad (2)$$

$$y_P = E\xi_P + F\eta_P + G\zeta_P + H, \quad (3)$$

where $A \dots D, E \dots H$ are the plate constants.

In principle, measurements of the (x_P, y_P) coordinates of 4 features would be sufficient to determine all plate constants. However, given various sources of error, it is important to carry out many measurements and to determine such quantities by a least-squares best fit (see below).

3. Plate measurements

☐ 5. Launch the VMA and prepare it for your measurements.

☐ Under **Configuration** (top left), enter the following coordinates and choices:

Observatory

Choose: Topocentric

Latitude: 38.3573°N Longitude: 26.7795°E

Country: GMT

Time Zone: Etc/GMT

Under **Display**, make choices that clearly allow you to see the features you are measuring. You may want to decrease the label density, for instance.

Under **Texture**, start with Phase without relief and make more sophisticated choices if your graphics card can handle the VMA at very high resolution, AstrolmageJ, and Jupyter running at the same time. Select LRO WAC Mosaic (bottom choice) but do feel free to explore other choices. When you move around any graphics objects, *be patient*. The VMA is showing you a mathematically calculated object and not just a photograph. When you move around or zoom in/out, your computer must do complex calculations.

Under **Overlay**, Start with No overlay (uncheck Show overlay). However, do experiment with other choices.

Under **Images**, Check for optional features. Be aware there is a wealth of imagery you can download as needed to complete lunar projects.

Close Configuration (choose OK)

☐ Go to **Lighting** .

Start with Penumbra and Specular in the 40-50% range.

Set both Diffuse and Smoothness and 100%.

Check Anti-alias.

☐ Go to **Information** .

☐ **6. Carry out a data file functionality check.**

Without closing any other applications, open Notepad and choose the test data file stored in folder /Measurements .

☐ Use the small data file indicated below, downloaded from my website under Measurements. Check that the syntax below produces results and that Wolfram Mathematica writes the output file. Check that the name and file location correctly match your case.

Notice that at the end of the cell, I have erased a ";" which would have silenced the output. That will be useful later, when you have many features in your own file.

```
In [57]: PlateMeasurements = Import["D:/AE-451/Project1/DataFiles/2023-04-13/Measurements
(* --- Reference dataset (professor's file) - do NOT overwrite ---
PlateMeasurements = Import["D:/AE-451/fabrizio-pinto-everybody-5980-04302020-
Print["Features loaded: ", Length[PlateMeasurements]];
Print["First feature : ", PlateMeasurements[[1]]];
```

Features loaded: 126

First feature : {Copernicus, 1094.78, 1519.61, -20.1, 9.621}

PASS 1 : Raw Pixel Coordinates

All cells below process the **raw** (uncorrected) measurement file.

You can isolate different variables from this entry by using the following syntax:

```
In [69]: PlateMeasurements[[1]]
PlateMeasurements[[1, 1]]
PlateMeasurements[[1, 2]]
PlateMeasurements[[1, 3]]
PlateMeasurements[[1, 4]]
PlateMeasurements[[1, 5]]
```

```
Out[69]: {Copernicus, 1094.78, 1519.61, -20.1, 9.621}
Copernicus
1094.78
1519.61
-20.1
9.621
```

As the number of features you are measuring increases, you can read them individually by just changing the value of the first index, K:

```
PlateMeasurements[[K, 3]]
```

In this sample file, you find only one feature. As you proceed with your work, monitor how your estimates and residuals behave as you increase the number of features considered. You must add 40-100 measurements carried out by you. Also **remember** that that two values (1321, 2710) do **not** apply to you. Measure Alphonsus again to obtain the pixel coordinates that are correct for you.

☐ If this test succeeded, change the name of the test data file to the following:

(this is what I would do if I were a student in AE-451 - change this to your name and to the image date!)

fabrizio-pinto-ae-451-project-1-2022-04-09-9405.txt

☐ **7. Open AstrolImageJ and record the date/time of the observation from the FITS header.**

☐ Open the FITS file in /ScienceFrames .

☐ Go to Tools, and use the Rotate Image options to place the North pole of the Moon up as closely as possible. Make sure that the *Mare Imbrrium* is at left/up and the lit part of the Moon is on the left with the terminator running vertically along the image.

Notice that two cursors, **Minimum** and **Maximum** , which you can move to optimize brightness and contrast during your measurements.

☐ At top right, notice the FITS HEADER. Observe and copy below (right in this cell) the value of the day/time variable (remember this is GMT):

```
DATE-OBS= '2022-04-xyT12:34:56
```

☐ **8. Set up the correct date/time in the VMA.**

At far right, go to **Ephemeris** , and enter the **same** Date and UTC Time shown by AstrolImageJ in the FITS header (DATE-OBS). Always remember to click on **Compute** .

☐ **9A. MEASURE THE PLATE.**

As we discussed in class several times, choose 40-100 features arranged **throughout the lunar disk**, carefully measure their pixel coordinates (round up the pixel coordinate to the nearest integer), and annotate the latitude and longitude in the same line in the text data file.

ATTENTION: If the feature has a composite name, such *Alphonsus B*, you MUST write it as AlphonsusB or the Jupyter Notebook will crash.

After you have measured at least 20 features, and every 20 features thereafter, carry out a check by reading out the data file with the same cell above (**PlateMeasurements = ...**) Remember to change the name of the file in that cell to the file name containing **your** name.

```
In [107... (* Change the first "1" to other values*)

PlateMeasurements[[99]]
PlateMeasurements[[99, 1]]
PlateMeasurements[[99, 2]]
PlateMeasurements[[99, 3]]
PlateMeasurements[[99, 4]]
PlateMeasurements[[99, 5]]

Out[107... {Wolf, 1189.6, 712.201, -16.633, -22.706}
Wolf
1189.6
712.201
-16.633
-22.706
```

The size of the sample is obtained. This number, N_f , must match the number of features you used. If it does not, you may have added blank lines or spaces at the bottom. **Delete any additional blanks.**

```
In [118... FeatureMax = Length[PlateMeasurements]

Out[118... 126
```

☐ **9B. SAVE THE VMA SCREEN.**

In the VMA, go to **File**, choose Save As, choose JPG and save the screen to:

D:\AE-451\Project1\DataFiles\2023-04-13\kaoutar-ammara-spring-2026-ae-451-project-1-2023-04-13-VMA.jpg

This file must be included in your submission package.

4. DATA FILE MANIPULATION AND FEATURE DISTRIBUTION

☐ **10. Execute all cells in this Section 4 to prepare your data for analysis.**

Now that all data have been read, the Cartesian coordinates (ξ_P, η_P, ζ_P) of every feature in the selenographic Cartesian frame are obtained from Eqs. (1). Let us use the information in the data file to assemble a list of all measured (X, Y) coordinates of the given features. We first create (Riffle) two flattened lists of the two coordinates and then we partition them into pairs so as to obtain one list (X_i, Y_i) coordinates for the i -th feature:

In [135...

```
PixelCoordinatesX = Part[PlateMeasurements[[1 ;; FeatureMax , 2]]];
PixelCoordinatesY = Part[PlateMeasurements[[1 ;; FeatureMax , 3]]];

Riffle[PixelCoordinatesX , PixelCoordinatesY];
PixelCoordinates = Partition[Riffle[PixelCoordinatesX , PixelCoordinatesY], 2];
```

The same procedure can be followed to create a list of all coordinates of features identified from a lunar atlas and whose coordinates, longitude λ and latitude β , both in degrees, are known:

In [144...

```
FeatureCoordinatesLong = Part[PlateMeasurements[[1 ;; FeatureMax , 4]]];
FeatureCoordinatesLat = Part[PlateMeasurements[[1 ;; FeatureMax , 5]]];

FeatureCoordinatesLongLat = Partition[Riffle[FeatureCoordinatesLong , FeatureCoordinatesLat], 2];
```

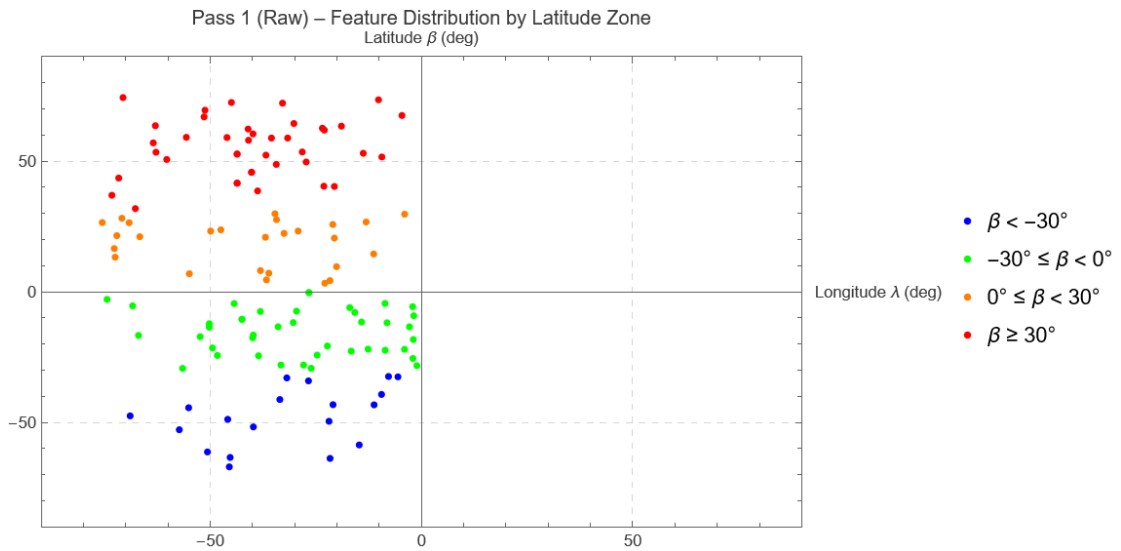
A distribution of these coordinates is provided by a (λ, β) -diagram. Notice that West longitude corresponds to $\lambda > 0^\circ$.

In [152...

```
(* Enhanced  $(\lambda, \beta)$ -distribution plot PASS 1 (Raw) *)
southFeatures = Select[FeatureCoordinatesLongLat, #[[2]] < -30 &];
midSFeatures = Select[FeatureCoordinatesLongLat, -30 <= #[[2]] < 0 &];
midNFeatures = Select[FeatureCoordinatesLongLat, 0 <= #[[2]] < 30 &];
northFeatures = Select[FeatureCoordinatesLongLat, #[[2]] >= 30 &];

LambdaBetaPlotRaw = ListPlot[
  {southFeatures, midSFeatures, midNFeatures, northFeatures},
  PlotRange -> {{-90, 90}, {-90, 90}},
  PlotStyle -> {Blue, Green, Orange, Red},
  PlotLegends -> {" $\beta < -30^\circ$ ", " $-30^\circ \leq \beta < 0^\circ$ ", " $0^\circ \leq \beta < 30^\circ$ ", " $\beta \geq 30^\circ$ "},
  AxesLabel -> {"Longitude  $\lambda$  (deg)", "Latitude  $\beta$  (deg)"},
  PlotLabel -> "Pass 1 (Raw) - Feature Distribution by Latitude Zone",
  GridLines -> Automatic,
  GridLinesStyle -> Directive[LightGray, Dashed],
  Frame -> True, ImageSize -> Large
];
LambdaBetaPlotRaw
```

Out[152...



In [159...

```
(* Export Pass 1 *)
```

```
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-r
```

```
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-r
```

Out[159...

```
D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-raw-lambda-beta-d\
```

```
> ist.pdf
```

```
D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-raw-lambda-beta-d\
```

```
> ist.png
```

The map above shows the distribution of features you chose. You can query the longitude and latitude of individual features to check that the list was properly manipulated and you obtain the correct values for every feature you used:

In [166...

```
FeatureCoordinatesLongLat;
FeatureCoordinatesLongLat[[1, 1]];
FeatureCoordinatesLongLat[[1, 2]];
```

Let us now assemble a list of the three dimensional coordinates in the local lunar Cartesian system, (ξ_i, η_i, ζ_i) for every such feature, by using the standard equations (Kopal1974, Eqs. (3.12)):

$$\xi_i = \cos \beta_i \sin \lambda_i,$$

$$\eta_i = \sin \beta_i,$$

$$\zeta_i = \cos \beta_i \cos \lambda_i,$$

where $\xi_i^2 + \eta_i^2 + \zeta_i^2 = 1$.

In [174...

```
(* Erase the ";" to understand what this process accomplishes. Replace the ";" w

SelenographicCartesian3D =
```

```
Table[{{Cos[FeatureCoordinatesLongLat[[i, 2]] Degree] Sin[FeatureCoordinatesLong
FeatureCoordinatesLongLat[[i, 2]] Degree] Cos[FeatureCoordinatesLongLat[[
```

The expression below is the standard Least Squares solution for the Multivariate Regression model in matrix language. Notice that this is done twice, once using the x -coordinates and then again with y -pixel coordinates. For an introductory but thorough description of this strategy, with examples, you are required to read and to reference: <https://online.stat.psu.edu/stat462/node/132/> . The following cell yields A , B , C , and D .

For instance, for the 1st feature in the list:

```
In [184... SelenographicCartesian3D[[1]]
Out[184... {-0.338826, 0.16713, 0.925886}
```

This list can be broken down into lists for each of the three components so as to produce a list of (ξ_i, η_i) coordinates only. In the absence of librations and neglecting the finite distance of the Moon from the observer, this would represent the position of the measured features on the surface:

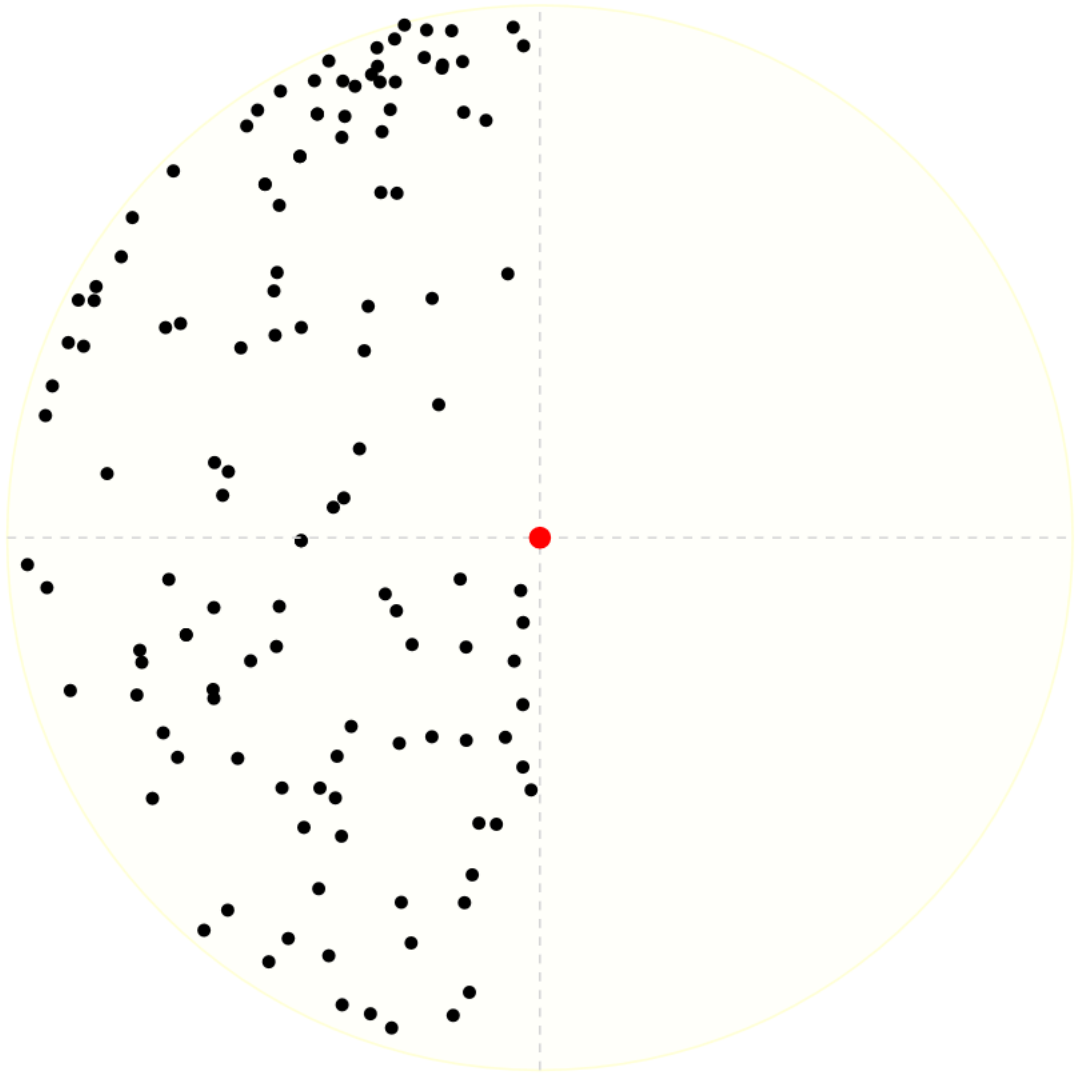
```
In [189... SelenographicCartesianxi = Part[SelenographicCartesian3D[[1 ;; FeatureMax , 1]]]
SelenographicCartesianeta = Part[SelenographicCartesian3D[[1 ;; FeatureMax , 2]]]
SelenographicCartesianzeta = Part[SelenographicCartesian3D[[1 ;; FeatureMax , 3]]]

SelenographicCartesian2D = Partition[Riffle[SelenographicCartesianxi, Selenograp
```

A diagram shows this distribution:

```
In [198... (* Enhanced 2-D selenographic distribution PASS 1 (Raw) *)
SelenographicCartesian2DPLLOT = ListPlot[
  SelenographicCartesian2D,
  PlotRange -> {{-1.1, 1.1}, {-1.1, 1.1}},
  AspectRatio -> 1.0,
  PlotStyle -> Directive[Black, PointSize[0.012]],
  AxesLabel -> {"ξ", "η"},
  PlotLabel -> "Pass 1 (Raw) - Measured Features on Lunar Disk (ξ, η)",
  GridLines -> Automatic,
  Frame -> True];
LunarDisk = Graphics[{Opacity[0.08, LightYellow], Disk[{0,0},1], Opacity[1],
LunarEquator = Graphics[{Dashed, LightGray, Line[{{-1,0},{1,0}}]}];
LunarMerid = Graphics[{Dashed, LightGray, Line[{{0,-1},{0,1}}]}];
LunarCenter = Graphics[{Red, PointSize[0.02], Point[{0,0}]}];
DiskPlotRaw = Show[LunarDisk, SelenographicCartesian2DPLLOT,
  LunarEquator, LunarMerid, LunarCenter, ImageSize -> 500];
DiskPlotRaw
```

Out[198...



In [206...

```
(* Export Pass 1: Lunar disk*)
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-r
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-r
```

Out[206...

```
D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-raw-xi-eta-disk.p\
```

```
> df
```

```
D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-raw-xi-eta-disk.p\
```

```
> ng
```

5. PLATE CALIBRATION

Let us now determine the Plate Constants. In order to prepare data as required by Mathematica (see for instance

<https://mathematica.stackexchange.com/questions/126283/performing-multiple-linear-regression-in-mathematica>), we use Riffle to create alternate records of the 4 quantities (ξ_i, η_i, ζ_i) and X_i , and then we flatten such list and partition it into separate $(\xi_i$

, η_i , ζ_i , X_i) records, and identically for $(\xi_i, \eta_i, \zeta_i, Y_i)$. In this way, we prepare the data for a linear model of the type required for a determination of the Plate Constants (Saunders 1900, p. 184):

$$x = A\xi + B\eta + C\zeta + D,$$

$$y = E\xi + F\eta + G\zeta + H,$$

```
In [213... PlateConstantsDataX = Partition[Flatten[Riffle[SelenographicCartesian3D, PixelCo
PlateConstantsDataY = Partition[Flatten[Riffle[SelenographicCartesian3D, PixelCo
```

☐ **11. Calculate the Plate Constants, (A, B, C, D, E, F, G, H) by a linear best fit:**

ATTENTION: You cannot carry out this operation if you have fewer than 8 measurements and, realistically, you need many more.

```
In [220... BestFitX = FindFit[PlateConstantsDataX, Dpc + Apc xi + Bpc eta + Cpc zeta, {Dpc,
BestFitY = FindFit[PlateConstantsDataY, Hpc + Epc xi + Fpc eta + Gpc zeta, {Hpc,
```

```
Out[220... {Dpc → 1516.14, Apc → 1468.58, Bpc → 29.2728, Cpc → 75.6608}
{Hpc → 1433.34, Epc → -34.9102, Fpc → 1462.12, Gpc → -182.671}
```

Additional information is available with the same syntax and data format by executing a Non-linear instruction, which in this case again corresponds to a linear fit. The residuals allow you to assess the goodness of fit.

```
In [226... BestFitXnlm = NonlinearModelFit[PlateConstantsDataX, Dpc + (Apc ) xi + (Bpc ) et

BestFitXnlm["BestFitParameters"];
BestFitXnlm[{"BestFit", "FitResiduals", "ParameterTable"}];
BestFitXnlm[{"FitResiduals"}];
BestFitXnlm[{"ParameterConfidenceIntervals"}];
BestFitXnlm["ParameterConfidenceIntervalTable"]
```

```
Out[226... FittedModel [ 1.52 × 103 + 29.3 eta + <<19>> xi + 75.7 zeta ]
```

	Estimate	Standard Error	Confidence Interval
Dpc	1516.14	6.39874	{1503.48, 1528.81}
Apc	1468.58	6.73601	{1455.24, 1481.91}
Bpc	29.2728	2.88809	{23.5555, 34.99}
Cpc	75.6608	7.29009	{61.2293, 90.0922}

```
In [233... BestFitYnlm = NonlinearModelFit[PlateConstantsDataY, Hpc + Epc xi + Fpc eta + G

BestFitYnlm["BestFitParameters"]
BestFitYnlm[{"BestFit", "FitResiduals", "ParameterTable"}];
BestFitYnlm[{"FitResiduals"}];
BestFitYnlm[{"ParameterConfidenceIntervals"}];
BestFitYnlm["ParameterConfidenceIntervalTable"];
```

Out[233...

FittedModel $\left[1.43 \times 10^3 + 1.46 \times 10^3 \text{ eta} - \ll 19 \gg \text{ xi} - 183. \text{ zeta} \right]$

{Hpc → 1433.34, Epc → -34.9102, Fpc → 1462.12, Gpc → -182.671}

Geometrical information available from the Plate Constants.

The plate constants have important geometric interpretations and yield additional information. For instance, the (X_C, Y_C) coordinates of the **center of the Moon in the plate**, in pixels, are the plate constants D and H .

☐ **12. Go to AstrolmageJ and verify the two values below are reasonable.**

In [247...

```
(* This is the position of the point of lat=0, Long=0*)
XOrigin = Normal[BestFitXnlm] /.{xi->0, eta ->0., zeta-> 1.0}
YOrigin = Normal[BestFitYnlm] /.{xi->0, eta ->0., zeta-> 1.0}
```

Out[247...

1591.8
1250.67

In [250...

```
XCenter = BestFitX[[1, 2]]
YCenter = BestFitY[[1, 2]]
```

Out[250...

1516.14
1433.34

The radius of the Moon, in pixels, along the two axes on the image is the scale factor given by the sum of the squares of the remaining three plate constants in each set, that is:

$$R_X^2 = A^2 + B^2 + C^2,$$

$$R_Y^2 = E^2 + F^2 + G^2.$$

☐ **13. Go to AstrolmageJ and verify the two values below are reasonable.**

In [260...

```
RadiusX = Sqrt[(BestFitX[[2, 2]])^2 + (BestFitX[[3, 2]])^2 + (BestFitX[[4, 2]])^2]
RadiusY = Sqrt[(BestFitY[[2, 2]])^2 + (BestFitY[[3, 2]])^2 + (BestFitY[[4, 2]])^2]
```

Out[260...

1470.82
1473.9

ATTENTION. Any difference between these two values must be attributed to both experimental error and possible field non-flatness. As one can see, these values coincide to within the order of magnitude of the fit residuals found. The average scale in km/pixel is, therefore, considering the average physical radius of the Moon (<https://nssdc.gsfc.nasa.gov/planetary/factsheet/moonfact.html>):

In [266...

```
AvePixRadius = ((RadiusX + RadiusY)/2)
```

```
AvePhysicalRadius = (1736.0 + 1738.1)/2
AvePixelScale = (AvePhysicalRadius)/(AvePixRadius)
```

Out[266... 1472.36
1737.05
1.17977

The geometric interpretation of the plate constants also allow us to determine the **orientation of the polar axis** of the Moon with respect to the vertical axis of the digital image (in degrees):

☐ **14. Go to AstrolmageJ and verify the value below is reasonable.**

```
In [277... BestFitX[[3, 2]]
BestFitY[[3, 2]]

ThetaRotObs = ArcTan[-BestFitX[[3, 2]]/BestFitY[[3, 2]]] (1/Degree)
```

Out[277... 29.2728
1462.12
-1.14695

6. Optical librations

The **optical libration angles** are revealed, therefore (Kopal1974, Eq. (3.15)), from a rotation in the opposite direction than the polar axis above:

```
In [284... AEUUnRot = RotationMatrix[-ThetaRotObs Degree].{BestFitX[[2, 2]]/RadiusX, BestFit
AEUUnRot[[1]]
ArcCos[AEUUnRot[[1]]] (1/Degree)
```

Out[284... {0.998752, -0.00369454}
0.998752
2.86309

☐ **15. Calculate the observed libration angles.**

The variable λ and β below are **your estimates of the libration angles**:

```
In [291... CGUnRot = RotationMatrix[-ThetaRotObs Degree].{BestFitX[[4, 2]]/RadiusX, BestFit

LambdaLibObs = -ArcSin[CGUnRot[[1]]] (1/Degree)
BetaLibObs = ArcSin[(-CGUnRot[[2]]/(Cos[-ArcSin[CGUnRot[[1]]]])] (1/Degree)
```

Out[291... {0.0539118, -0.122883}
-3.09042
7.06883

With the available data, we have the following **libration estimates obtained from observation**, expressed in decimal degrees.

(The DMS values below are computed from the decimal output of the cells above.)

Libration in Longitude: $\lambda_{\text{lib,obs}} = -3.09042^\circ \approx -3^\circ 05'$

Libration in Latitude: $\beta_{\text{lib,obs}} = +7.06883^\circ \approx +7^\circ 04'$

(Always re-read `N[LambdaLibObs]` and `N[BetaLibObs]` from the cell output above and recompute DMS as `IntegerPart[val]°` and `Round[60 Abs[FractionalPart[val]]]'`.)

```
In [298... (* Preserve Pass 1 (Raw) results under unique variable names *)
LambdaLibObsResultRaw = LambdaLibObs;
BetaLibObsResultRaw   = BetaLibObs;
ThetaRotObsResultRaw  = ThetaRotObs;
RadiusXResultRaw      = RadiusX;
RadiusYResultRaw      = RadiusY;
AvePixScaleResultRaw  = AvePixelScale;
RMSXResultRaw         = Sqrt[Mean[BestFitXnlm["FitResiduals"]^2]];
RMSYResultRaw         = Sqrt[Mean[BestFitYnlm["FitResiduals"]^2]];
Print["Pass 1 (Raw) results saved as *ResultRaw"];
```

Pass 1 (Raw) results saved as *ResultRaw

7. Comparison with published data

☐ 15. Record the libration angles from the VMA.

This data must be compared to the **libration values given by the VMA** for the date/time given in the FITS header at the DATE-OBS entry.

Below, reproduce the DATE-OBS line from the FITS header and the copy the libration angles given at `Ephemeris` for the exact time of this observation:

The VMA provides:

$$l_{0,\text{VMA}} = -2^\circ 42' = -2.70^\circ,$$

$$b_{0,\text{VMA}} = 7^\circ 30' = 7.50^\circ.$$

☐ 16. Save the VMA ephemeris.

In the VMA, go to `File`, choose `Save ephemeris to file`, change no options, and save the file to subfolder:

C:/AE-451/DataFiles/2023-04-13/Measurements

with file name:

(this is what I would do if I were a student in AE-451 - change this to your name!)

fabrizio-pinto-ae-451-project-1-2022-04-09-ephem-VMA.txt

In Project 2, you will learn how to produce an ephemeris file directly from the NASA/JPL database.

☐ **17. Record the absolute libration angle errors.**

Calculate and indicate below the **absolute errors of your libration estimates**:

$$\Delta\lambda = l_{0,\text{VMA}} - \lambda_{\text{lib,obs}} ,$$

$$\Delta\beta = b_{0,\text{VMA}} - \beta_{\text{lib,obs}} .$$

Project 1 Reporting activities

Before the announced deadline, prepare your work for submission as follows.

Click into the empty cell below, go to Edit above, choose **Insert Image** and include:

☐ **3. YOUR VMA screenshot**

Furthermore, when you upload this Jupyter Notebook to Blackboard, also separately upload the following files:

☐ **4. YOUR DATA FILE listing all the features you used.**

☐ **5. The VMA Ephemeris FILE.**

☐ **6. YOUR SLIDE file, in pdf format**

Finally, go to File above, choose **Download As** *Notebook (.ipynb)* and download this completed Notebook to

☐ **1. AE-451/Projects/Project-1/fabrizio-pinto-spring-2023-ae-451-project-1.ipynb**

Repeat the same process saving the file to:

☐ **2. AE-451/Projects/Project-1/fabrizio-pinto-spring-2023-ae-451-project-1.html**

After saving this latter file to html, open it and print it to PDF. **Submit the pdf file.**

Obviously, change these filenames to your name.

☐ **IN TOTAL, YOU MUST UPLOAD six (6) FILES TO BLACKBOARD BEFORE THE DEADLINE TO RECEIVE FULL CREDIT FOR THIS PROJECT 1.**

I reserve the right to ask you to show me **ANY** Project 1 information at any time, including during your presentation.

Selected References

The Reference list below is absolutely not exhaustive and it only represents a small selection of all published works on selenography chosen by me for the purpose to bring to your attention treatments that are representative of the literature, easily and freely reachable, and at your level.

1. S. A. Saunder, The determination of selenographic positions and the measurement of lunar photographs, *MNRAS*, **60**, 174-201 (1900).
<https://academic.oup.com/mnras/article/60/3/174/983575>
2. Z. Kopal, Topography of the Moon, *Space Science Reviews*, **4**, 737-855 (1965).
<https://adsabs.harvard.edu/full/1965SSRv....4..737K>
3. Z. Kopal and R. W. Carder, *Mapping of the Moon* (Springer, Dordrecht, 1974), Ch. 3.
4. J. Giesen, <http://www.jgiesen.de/moonlibration/>
5. Fabrizio Pinto, Study of the determination of the topocentric lunar librations by a best fit estimate of the plate constants from digital images with a small telescope, *Proceedings for the 44th Annual Conference of the Society for Astronomical Sciences*, pp 25-32 (2025).
https://socastrosci.org/wp-content/uploads/2025/07/2025-Proceedings_Ver1.2a.pdf

Appendix 1: Correction for a finite distance of the observer

Here we implement corrections to the measured plate coordinates for the case of a non-infinite Earth, following the procedure discussed by Saunder (§ 4 and §8), with slight modifications in the process and notation.

Unlike in Saunder's article, we shall not set the radius of the Moon equal to unity (§ 4) but to its physical value R_M to retain manifestly correct units, so that the angle s' subtended by the Moon, assumed at a distance r , will be:

$$\sin s' = \frac{R_M}{r}. \quad (\text{A1})$$

With reference to Saunder's (Fig. 2), we redefine as follows (refer also to our definitions above):

$$\overline{MR} = \rho, \quad \overline{MQ} = \rho + \delta\rho, \quad \overline{PR} = z = \zeta R_M = \sqrt{R_M^2 - \rho^2},$$

and the value of z for which the ordinate through Q meets the sphere, $z + \delta z$:

$$z + \delta z = (\zeta + \delta\zeta) R_M = \sqrt{R_M^2 - (\rho + \delta\rho)^2}.$$

Squaring and expanding to 1st order:

$$(z + \delta z)^2 \simeq z + 2z\delta z \simeq (\zeta + 2\zeta\delta\zeta) R_M^2 = R_M^2 \left[1 - \left(\frac{\rho + \delta\rho}{R_M} \right)^2 \right] \simeq R_M^2 \left[1 - \left(\frac{\rho^2 + 2\rho\delta\rho}{R_M^2} \right) \right]$$

Identifying the terms in 1st order, we find:

$$z\delta z = -\rho\delta\rho.$$

Compare to Kopal (1965, p 7 PDF):

$$\delta = \frac{\overline{MD} \overline{PD}}{\overline{MP}} \sin s' = \frac{\rho(z + \delta z)}{R_M} \sin s'$$

Now we have the information needed to correct the positions on the disk and obtain the orthographic projections of all feature locations (Kopal(1965, p 743).

Consider for instance the first feature in the list:

```
In [336... PlateMeasurements[[1]]
PlateMeasurements[[1, 1]]
PlateMeasurements[[1, 2]]
PlateMeasurements[[1, 3]]
PlateMeasurements[[1, 4]]
PlateMeasurements[[1, 5]]

Out[336... {Copernicus, 1094.78, 1519.61, -20.1, 9.621}
Copernicus
1094.78
1519.61
-20.1
9.621
```

The radius of the Moon was measured, in these same pixel units, as:

```
In [346... AvePixRadius

Out[346... 1472.36
```

The coordinates of the center of the Moon are:

```
In [351... XCenter
YCenter

Out[351... 1516.14
1433.34
```

Therefore, the uncorrected coordinates (x', y', z') with respect to the lunar center in the image, expressed in such a way that $x'^2 + y'^2 + z'^2 = 1$, are:

```
In [357... xpr = ((PlateMeasurements[[1, 2]] - XCenter)/AvePixRadius)
ypr = ((PlateMeasurements[[1, 3]] - YCenter)/AvePixRadius)

zpr = Sqrt[1-((PlateMeasurements[[1, 2]] - XCenter)/AvePixRadius)^2-((PlateMeasu

Out[357... -0.286186
0.0585919
0.956381
```

According to the ephemeris, the distance of the Moon at the epoch of the photograph was, in meters:

In [364...

```
(* Moon-Earth distance at the epoch of the photograph, in metres.
Source: Virtual Moon Atlas (VMA) Ephemeris
Date : 2023-04-13 03:10:47 UTC
Observatory: +38°22' N, E26°47', TZ 0h
VMA Distance: 368567 km ← hardcoded below *)

rMoon = 368567 10^3; (* metres - from VMA ephemeris *)
Print["Moon distance: ", rMoon/10^3, " km"];
```

Moon distance: 368567 km

VMA Ephemeris Results for This Image

The following values were read directly from the **Virtual Moon Atlas** Ephemeris tab for the observation epoch **2023-04-13 03:10:47 UTC** at the observatory coordinates **+38°22' N, E26°47', TZ 0h**.

Quantity	Value
Distance	368 567 km
Apparent diameter	32.42'
Phase	273.4°
Lunation	22.41 days
Illumination	53.0 %
Colongitude	179.3°
Sub-solar latitude	-0.3°
Libration in Longitude	-02°42' = -2.70°
Libration in Latitude	+07°30' = +7.50°
Position angle	-9.1°
Azimuth	+164°02'
Altitude	+21°55'

These are the **reference libration values** you will compare your photogrammetric results against at the end of the notebook.

In [371...

```
(* Libration in Longitude: -02 deg 42 min *)
LambdaLibVMA = -(2 + 42/60); (* = -2.70 deg *)

(* Libration in Latitude: +07 deg 30 min *)
BetaLibVMA = +(7 + 30/60); (* = +7.50 deg *)

(* Position angle of the axis *)
PositionAngleVMA = -9.1; (* deg *)

Print["VMA Libration in Longitude : ", N[LambdaLibVMA], " deg"];
Print["VMA Libration in Latitude : ", N[BetaLibVMA], " deg"];
Print["VMA Position Angle : ", PositionAngleVMA, " deg"];
```

VMA Libration in Longitude : -2.7 deg
 VMA Libration in Latitude : 7.5 deg
 VMA Position Angle : -9.1 deg

Given the average physical radius used herein, the apparent radius is, in radians and degrees, respectively:

```
In [386... AvePhysicalRadius 10^3/rMoon

(AvePhysicalRadius 10^3/rMoon)^2

AveAngularRadius = ArcSin[AvePhysicalRadius 10^3/rMoon]
AveAngularRadiusDeg = ArcSin[AvePhysicalRadius 10^3/rMoon] (180/Pi)
```

```
Out[386... 0.00471298
0.0000222122
0.004713
0.270035
```

The correction is therefore, in this case, in units in which the lunar radius is equal to unity and to its pixel value, respectively:

```
In [394... deltaX = xpr zpr Sin[AveAngularRadius]
deltaXpix = xpr zpr Sin[AveAngularRadius] AvePixRadius

deltaY = ypr zpr Sin[AveAngularRadius]
deltaYpix = ypr zpr Sin[AveAngularRadius] AvePixRadius
```

```
Out[394... -0.00128996
-1.89928
0.000264098
0.388846
```

Let us process all data and generate a new file containing corrected coordinates.

The first step is to construct a table with uncorrected data expressed with respect to the center of the Moon on the image:

```
In [402... PlateMeasurementsCentered = Table[{PlateMeasurements[[i, 1]], ((PlateMeasurements
```

```
In [403... Export["D:/AE-451/Project1/DataFiles/2023-04-13/Measurements/kaoutar-ammara-spri
```

```
Out[403... D:/AE-451/Project1/DataFiles/2023-04-13/Measurements/kaoutar-ammara-spring-2026-ae-451-p\
```

```
> roject-1-centered.txt
```

Testing the above algorithm on this file should yield no change in the results except for the fact that the coordinates of the center will be at (0, 0).

Let us now create the corrected file:

```
In [412... PlateMeasurementsCenteredCorrections = Table[{PlateMeasurements[[i, 1]], -(PlateMeasurementsCenteredCorrections[[i, 1]] + PlateMeasurementsCenteredCorrections[[i, 1]])}];

In [413... Export["D:/AE-451/Project1/DataFiles/2023-04-13/Measurements/kaoutar-ammara-spring-2026-ae-451-p\

Out[413... D:/AE-451/Project1/DataFiles/2023-04-13/Measurements/kaoutar-ammara-spring-2026-ae-451-p\

> roject-1-corrections.txt
```

```
In [414... PlateMeasurementsCenteredCorrected = Table[{PlateMeasurements[[i, 1]], (PlateMeasurementsCenteredCorrected[[i, 1]] + PlateMeasurementsCenteredCorrected[[i, 1]])}];

In [415... Export["D:/AE-451/Project1/DataFiles/2023-04-13/Measurements/kaoutar-ammara-spring-2026-ae-451-p\

Out[415... D:/AE-451/Project1/DataFiles/2023-04-13/Measurements/kaoutar-ammara-spring-2026-ae-451-p\

> roject-1-centered-corr.txt
```

Now we are ready to process the corrected coordinates.

PASS 2 — Centered Pixel Coordinates

The centered file shifts pixel coordinates so the Moon's centre lies at (0, 0).
The plate constants D and H should therefore be ≈ 0 for this pass.

```
In [428... (*second pass: centered coordinates.*)
PlateMeasurements = Import["D:/AE-451/Project1/DataFiles/2023-04-13/Measurements/kaoutar-ammara-spring-2026-ae-451-p\
```

Pass2 Feature count & spot-check

```
In [434... FeatureMax = Length[PlateMeasurements]

Out[434... 126

In [435... (* Spot-check: feature 99 *)
PlateMeasurements[[99]]

Out[435... {Wolf, -326.547, -721.141, -16.633, -22.706}
```

— Pass2 Pixel & atlas coordinates

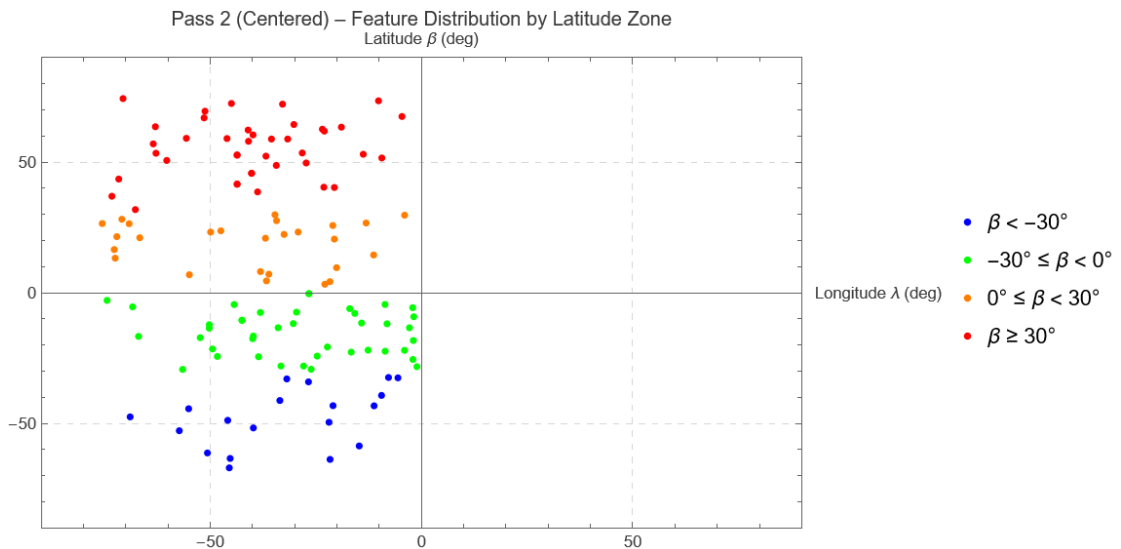
```
In [441... PixelCoordinatesX = Part[PlateMeasurements[[1 ;; FeatureMax, 2]]];
PixelCoordinatesY = Part[PlateMeasurements[[1 ;; FeatureMax, 3]]];
PixelCoordinates = Partition[Riffle[PixelCoordinatesX, PixelCoordinatesY], 2];
FeatureCoordinatesLong = Part[PlateMeasurements[[1 ;; FeatureMax, 4]]];
FeatureCoordinatesLat = Part[PlateMeasurements[[1 ;; FeatureMax, 5]]];
FeatureCoordinatesLongLat = Partition[Riffle[FeatureCoordinatesLong, FeatureCoordinatesLat], 2];
```

— Pass2 (λ, β) distribution plot

In [451...

```
(* ( $\lambda, \beta$ )-distribution - Pass 2 (Centered) *)
southFeatures = Select[FeatureCoordinatesLongLat, #[[2]] < -30 &];
midSFeatures  = Select[FeatureCoordinatesLongLat, -30 <= #[[2]] < 0 &];
midNFeatures  = Select[FeatureCoordinatesLongLat, 0 <= #[[2]] < 30 &];
northFeatures = Select[FeatureCoordinatesLongLat, #[[2]] >= 30 &];
LambdaBetaPlotPass2 = ListPlot[
  {southFeatures, midSFeatures, midNFeatures, northFeatures},
  PlotRange -> {{-90, 90}, {-90, 90}},
  PlotStyle -> {Blue, Green, Orange, Red},
  PlotLegends -> {" $\beta < -30^\circ$ ", " $-30^\circ \leq \beta < 0^\circ$ ", " $0^\circ \leq \beta < 30^\circ$ ", " $\beta \geq 30^\circ$ "},
  AxesLabel -> {"Longitude  $\lambda$  (deg)", "Latitude  $\beta$  (deg)"},
  PlotLabel -> "Pass 2 (Centered) - Feature Distribution by Latitude Zone",
  GridLines -> Automatic,
  GridLinesStyle -> Directive[LightGray, Dashed],
  Frame -> True, ImageSize -> Large
];
LambdaBetaPlotPass2
```

Out[451...



In [458...

```
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-c
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-c
```

Out[458...

D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-centered-lambda-b\

> eta-dist.pdf

D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-centered-lambda-b\

> eta-dist.png

— Pass2 Selenographic 3D → 2D Cartesian

In [465...

```
SelenographicCartesian3D = Table[
  {Cos[FeatureCoordinatesLongLat[[i,2]] Degree] Sin[FeatureCoordinatesLongLat[[i,
  Sin[FeatureCoordinatesLongLat[[i,2]] Degree],
```

```

Cos[FeatureCoordinatesLongLat[[i,2]] Degree] Cos[FeatureCoordinatesLongLat[[i
{i, 1, FeatureMax}]];
SelenographicCartesian3D[[1]]
SelenographicCartesianxi = Part[SelenographicCartesian3D[[1 ;; FeatureMax, 1]]
SelenographicCartesianeta = Part[SelenographicCartesian3D[[1 ;; FeatureMax, 2]]
SelenographicCartesianzeta = Part[SelenographicCartesian3D[[1 ;; FeatureMax, 3]]
SelenographicCartesian2D = Partition[Riffle[SelenographicCartesianxi, Selenogr

```

Out[465... {-0.338826, 0.16713, 0.925886}

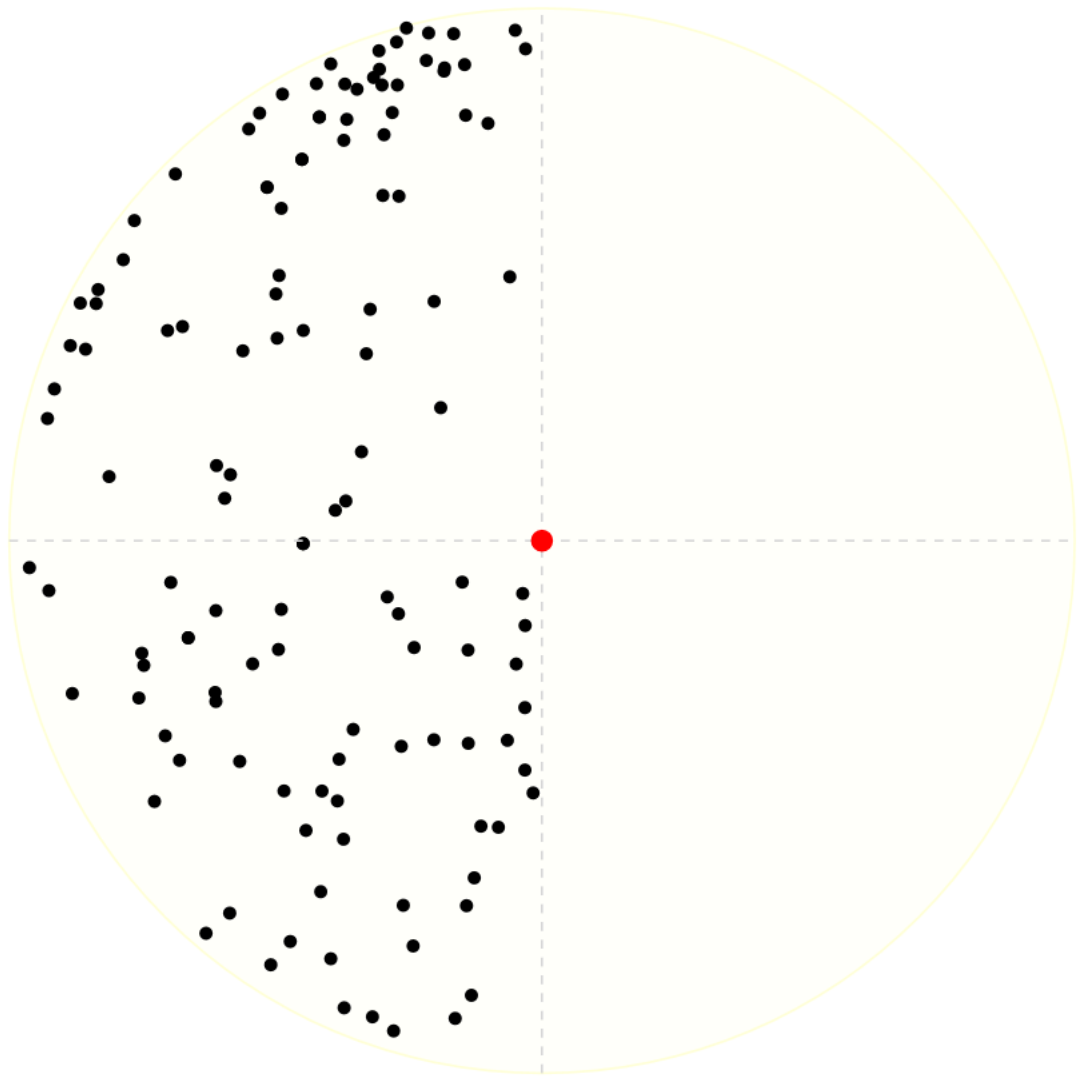
— Pass2 Lunar disk (ξ , η) plot

```

In [475... (* 2-D selenographic distribution - Pass 2 (Centered) *)
SelenographicCartesian2DPLOT = ListPlot[
  SelenographicCartesian2D,
  PlotRange -> {{-1.1, 1.1}, {-1.1, 1.1}}, AspectRatio -> 1.0,
  PlotStyle -> Directive[Black, PointSize[0.012]],
  AxesLabel -> {" $\xi$ ", " $\eta$ "},
  PlotLabel -> "Pass 2 (Centered) - Measured Features on Lunar Disk ( $\xi$ ,  $\eta$ )",
  GridLines -> Automatic, Frame -> True];
LunarDisk = Graphics[{Opacity[0.08, LightYellow], Disk[{0,0},1], Opacity[1],
LunarEquator = Graphics[{Dashed, LightGray, Line[{{-1,0},{1,0}}]}];
LunarMerid = Graphics[{Dashed, LightGray, Line[{{0,-1},{0,1}}]}];
LunarCenter = Graphics[{Red, PointSize[0.02], Point[{0,0}]}];
DiskPlotPass2 = Show[LunarDisk, SelenographicCartesian2DPLOT,
  LunarEquator, LunarMerid, LunarCenter, ImageSize -> 500];
DiskPlotPass2

```


Out[475...



In [483...

```
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-c
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-c
```

Out[483...

```
D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-centered-xi-eta-d\
```

```
> isk.pdf
```

```
D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-centered-xi-eta-d\
```

```
> isk.png
```

— Pass2 Plate calibration (FindFit + NonlinearModelFit)

In [490...

```
PlateConstantsDataX = Partition[Flatten[Riffle[SelenographicCartesian3D, PixelCo
PlateConstantsDataY = Partition[Flatten[Riffle[SelenographicCartesian3D, PixelCo

BestFitX = FindFit[PlateConstantsDataX, Dpc + Apc xi + Bpc eta + Cpc zeta,
  {Dpc, Apc, Bpc, Cpc}, {xi, eta, zeta}]
BestFitY = FindFit[PlateConstantsDataY, Hpc + Epc xi + Fpc eta + Gpc zeta,
  {Hpc, Epc, Fpc, Gpc}, {xi, eta, zeta}]
```

Out[490...] $\{Dpc \rightarrow -1.57419 \times 10^{-13}, Apc \rightarrow 1468.58, Bpc \rightarrow 29.2728, Cpc \rightarrow 75.6608\}$
 $\{Hpc \rightarrow -2.2629 \times 10^{-13}, Epc \rightarrow -34.9102, Fpc \rightarrow 1462.12, Gpc \rightarrow -182.671\}$

In [494...] `BestFitXnlm = NonlinearModelFit[PlateConstantsDataX,
 Dpc + Apc xi + Bpc eta + Cpc zeta, {Dpc, Apc, Bpc, Cpc}, {xi, eta, zeta}];
 BestFitYnlm = NonlinearModelFit[PlateConstantsDataY,
 Hpc + Epc xi + Fpc eta + Gpc zeta, {Hpc, Epc, Fpc, Gpc}, {xi, eta, zeta}];
 BestFitXnlm["ParameterConfidenceIntervalTable"]
 BestFitYnlm["ParameterConfidenceIntervalTable"]`

Out[494...]

	Estimate	Standard Error	Confidence Interval
Dpc	-1.57419×10^{-13}	6.39874	{-12.6669, 12.6669}
Apc	1468.58	6.73601	{1455.24, 1481.91}
Bpc	29.2728	2.88809	{23.5555, 34.99}
Cpc	75.6608	7.29009	{61.2293, 90.0922}
	Estimate	Standard Error	Confidence Interval
Hpc	-2.2629×10^{-13}	8.53903	{-16.9039, 16.9039}
Epc	-34.9102	8.98912	{-52.7051, -17.1153}
Fpc	1462.12	3.85412	{1454.49, 1469.75}
Gpc	-182.671	9.72853	{-201.93, -163.413}

— Pass2 Geometric quantities from plate constants

In [502...] `XCenter = BestFitX[[1, 2]]
 YCenter = BestFitY[[1, 2]]
 RadiusX = Sqrt[(BestFitX[[2, 2]])^2 + (BestFitX[[3, 2]])^2 + (BestFitX[[4, 2]])^2]
 RadiusY = Sqrt[(BestFitY[[2, 2]])^2 + (BestFitY[[3, 2]])^2 + (BestFitY[[4, 2]])^2]
 AvePixRadius = (RadiusX + RadiusY) / 2
 AvePhysicalRadius = (1736.0 + 1738.1) / 2
 AvePixelScale = AvePhysicalRadius / AvePixRadius
 ThetaRotObs = ArcTan[-BestFitX[[3, 2]] / BestFitY[[3, 2]]] (1/Degree)`

Out[502...]

-13
 -1.57419×10^{-13}
 -13
 -2.2629×10^{-13}
 1470.82
 1473.9
 1472.36
 1737.05
 1.17977
 -1.14695

— Pass2 Optical libration angles

In [514...] `AEUnRot = RotationMatrix[-ThetaRotObs Degree].{BestFitX[[2, 2]]/RadiusX,
 BestFitY[[2, 2]]/RadiusY}`

```

AEUnRot[[1]]
ArcCos[AEUnRot[[1]]] (1/Degree)

CGUnRot      = RotationMatrix[-ThetaRotObs Degree].{BestFitX[[4,2]]/RadiusX,
                                                    BestFitY[[4,2]]/RadiusY}

LambdaLibObs = -ArcSin[CGUnRot[[1]]] (1/Degree)
BetaLibObs   = ArcSin[-CGUnRot[[2]] / Cos[-ArcSin[CGUnRot[[1]]]]] (1/Degree)

```

```

Out[514...] {0.998752, -0.00369454}
0.998752
2.86309
{0.0539118, -0.122883}
-3.09042
7.06883

```

```

In [521...] Print["Pass 2 (Centered) – Libration Results"];
Print["  λ_lib,obs = ", N[LambdaLibObs, 7], " deg"];
Print["  β_lib,obs = ", N[BetaLibObs, 7], " deg"];
Print["  θ_rot      = ", N[ThetaRotObs, 7], " deg"];

```

```

Pass 2 (Centered) -- Libration Results
\[Lambda]_lib,obs = -3.09042 deg
\[Beta]_lib,obs   = 7.06883 deg
\[Theta]_rot      = -1.14695 deg

```

```

In [525...] LambdaLibObsResultPass2 = LambdaLibObs;
BetaLibObsResultPass2   = BetaLibObs;
ThetaRotObsResultPass2  = ThetaRotObs;
RadiusXResultPass2      = RadiusX;
RadiusYResultPass2      = RadiusY;
AvePixScaleResultPass2  = AvePixelScale;
RMSXResultPass2         = Sqrt[Mean[BestFitXnlm["FitResiduals"]^2]];
RMSYResultPass2         = Sqrt[Mean[BestFitYnlm["FitResiduals"]^2]];
Print["Results stored as *ResultPass2"];

```

```
Results stored as *ResultPass2
```

PASS 3 Parallax-Corrected Coordinates

The corrected file applies the finite-observer distance correction (Appendix 1).
Compare the libration angles from this pass to the VMA reference values below.

```

In [539...] (*third pass: parallax-corrected coordinates.*)
PlateMeasurements = Import["D:/AE-451/Project1/DataFiles/2023-04-13/Measurements

```

— Pass3 Feature count & spot-check

```

In [545...] FeatureMax = Length[PlateMeasurements]

```

```

Out[545...] 126

```

```

In [546...] (* Spot-check: feature 99 *)
PlateMeasurements[[99]]

```

Out[546... {Wolf, -325.249, -718.275, -16.633, -22.706}

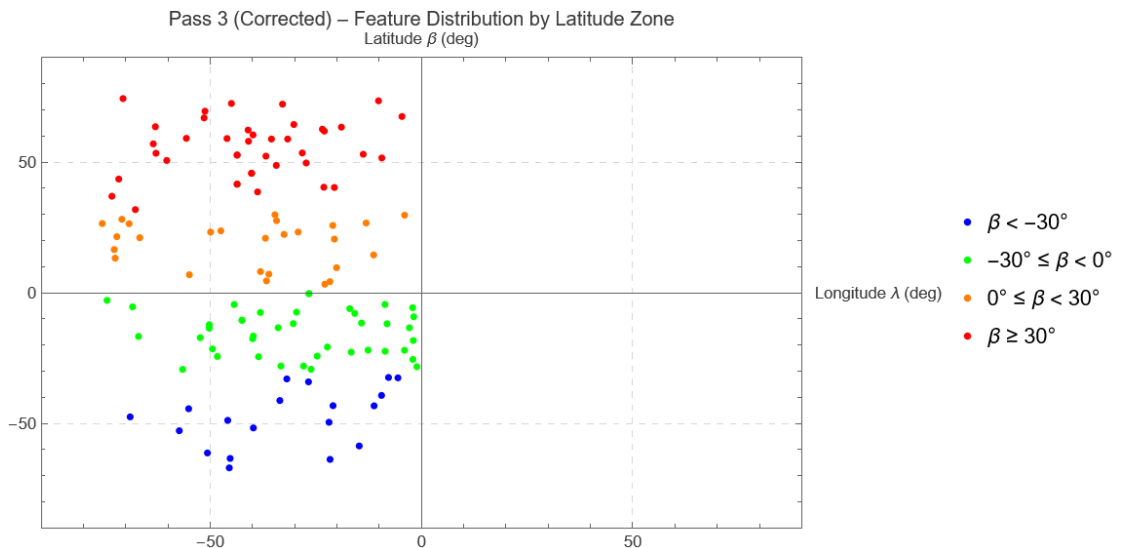
— Pass3 Pixel & atlas coordinates

```
In [552... PixelCoordinatesX = Part[PlateMeasurements[[1 ;; FeatureMax, 2]]];
PixelCoordinatesY = Part[PlateMeasurements[[1 ;; FeatureMax, 3]]];
PixelCoordinates = Partition[Riffle[PixelCoordinatesX, PixelCoordinatesY], 2];
FeatureCoordinatesLong = Part[PlateMeasurements[[1 ;; FeatureMax, 4]]];
FeatureCoordinatesLat = Part[PlateMeasurements[[1 ;; FeatureMax, 5]]];
FeatureCoordinatesLongLat = Partition[Riffle[FeatureCoordinatesLong, FeatureCoordinatesLat], 2];
```

Pass3 (λ, β) distribution plot

```
In [562... (* ( $\lambda, \beta$ )-distribution - Pass 3 (Corrected) *)
southFeatures = Select[FeatureCoordinatesLongLat, #[[2]] < -30 &];
midSFeatures = Select[FeatureCoordinatesLongLat, -30 <= #[[2]] < 0 &];
midNFeatures = Select[FeatureCoordinatesLongLat, 0 <= #[[2]] < 30 &];
northFeatures = Select[FeatureCoordinatesLongLat, #[[2]] >= 30 &];
LambdaBetaPlotPass3 = ListPlot[
  {southFeatures, midSFeatures, midNFeatures, northFeatures},
  PlotRange -> {{-90, 90}, {-90, 90}},
  PlotStyle -> {Blue, Green, Orange, Red},
  PlotLegends -> {" $\beta < -30^\circ$ ", " $-30^\circ \leq \beta < 0^\circ$ ", " $0^\circ \leq \beta < 30^\circ$ ", " $\beta \geq 30^\circ$ "},
  AxesLabel -> {"Longitude  $\lambda$  (deg)", "Latitude  $\beta$  (deg)"},
  PlotLabel -> "Pass 3 (Corrected) - Feature Distribution by Latitude Zone",
  GridLines -> Automatic,
  GridLinesStyle -> Directive[LightGray, Dashed],
  Frame -> True, ImageSize -> Large
];
LambdaBetaPlotPass3
```

Out[562...



```
In [569... Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-c
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-c
```

```

Out[569... D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-corrected-lambda-\\

> beta-dist.pdf

D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-corrected-lambda-\\

> beta-dist.png

```

Pass3 Selenographic 3D → 2D Cartesian

```

In [576... SelenographicCartesian3D = Table[
  {Cos[FeatureCoordinatesLongLat[[i,2]] Degree] Sin[FeatureCoordinatesLongLat[[i
    Sin[FeatureCoordinatesLongLat[[i,2]] Degree],
    Cos[FeatureCoordinatesLongLat[[i,2]] Degree] Cos[FeatureCoordinatesLongLat[[i
  {i, 1, FeatureMax}]];
SelenographicCartesian3D[[1]]
SelenographicCartesianxi = Part[SelenographicCartesian3D[[1 ;; FeatureMax, 1]]
SelenographicCartesianeta = Part[SelenographicCartesian3D[[1 ;; FeatureMax, 2]]
SelenographicCartesianzeta = Part[SelenographicCartesian3D[[1 ;; FeatureMax, 3]]
SelenographicCartesian2D = Partition[Riffle[SelenographicCartesianxi, Selenogr

Out[576... {-0.338826, 0.16713, 0.925886}

```

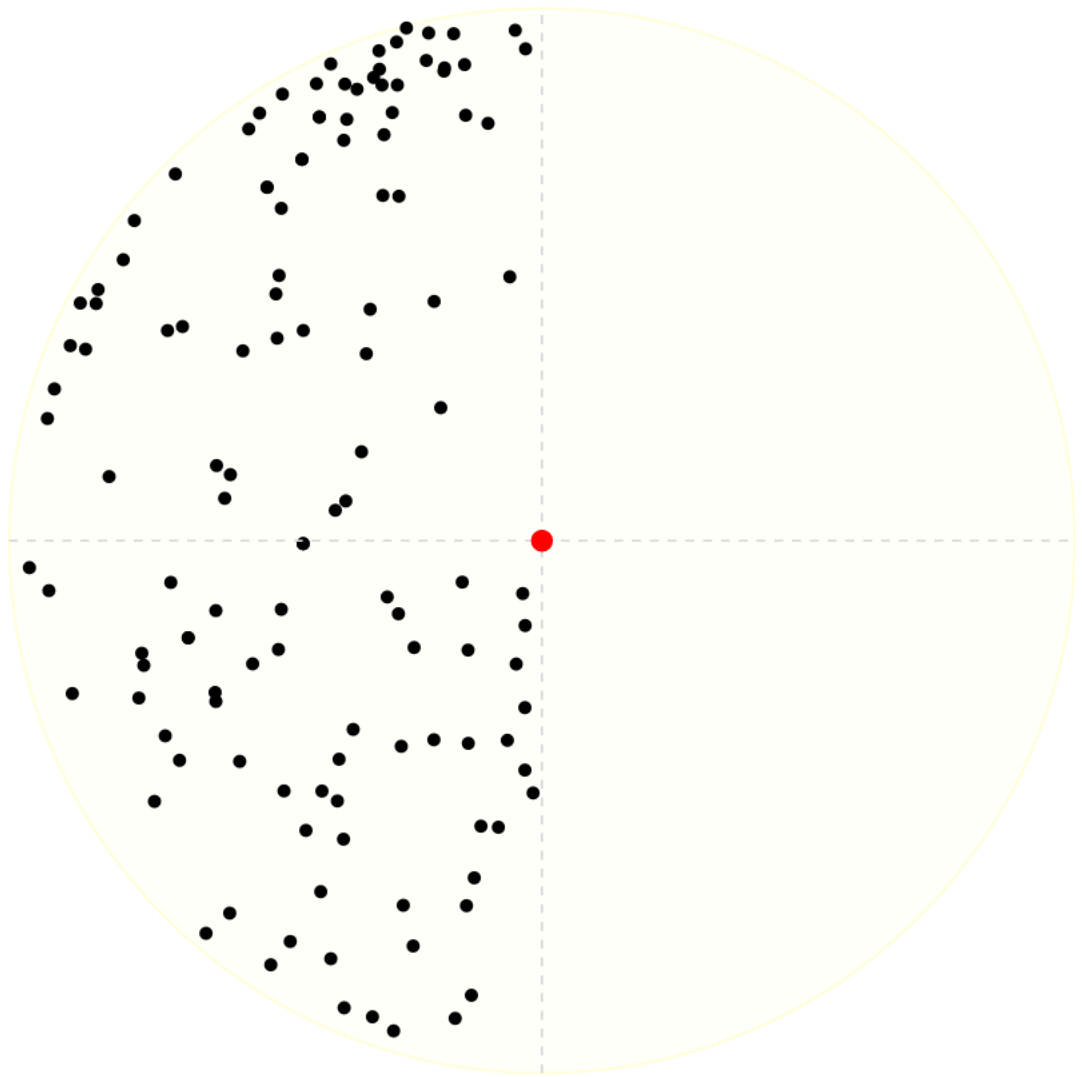
Pass3 Lunar disk (ξ , η) plot

```

In [586... SelenographicCartesian2DPlot = ListPlot[
  SelenographicCartesian2D,
  PlotRange -> {{-1.1, 1.1}, {-1.1, 1.1}}, AspectRatio -> 1.0,
  PlotStyle -> Directive[Black, PointSize[0.012]],
  AxesLabel -> {" $\xi$ ", " $\eta$ "},
  PlotLabel -> "Pass 3 (Corrected) - Measured Features on Lunar Disk ( $\xi$ ,  $\eta$ )",
  GridLines -> Automatic, Frame -> True];
LunarDisk = Graphics[{Opacity[0.08, LightYellow], Disk[{0,0},1], Opacity[1],
LunarEquator = Graphics[{Dashed, LightGray, Line[{{-1,0},{1,0}}]}];
LunarMerid = Graphics[{Dashed, LightGray, Line[{{0,-1},{0,1}}]}];
LunarCenter = Graphics[{Red, PointSize[0.02], Point[{0,0}]}];
DiskPlotPass3 = Show[LunarDisk, SelenographicCartesian2DPlot,
  LunarEquator, LunarMerid, LunarCenter, ImageSize -> 500];
DiskPlotPass3

```

Out[586...



In [594...

```
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-c
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-c
```

Out[594...

```
D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-corrected-xi-eta-\
```

```
> disk.pdf
```

```
D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-corrected-xi-eta-\
```

```
> disk.png
```

— Pass3 Plate calibration (FindFit + NonlinearModelFit)

In [601...

```
PlateConstantsDataX = Partition[Flatten[Riffle[SelenographicCartesian3D, PixelCo
PlateConstantsDataY = Partition[Flatten[Riffle[SelenographicCartesian3D, PixelCo

BestFitX = FindFit[PlateConstantsDataX, Dpc + Apc xi + Bpc eta + Cpc zeta,
  {Dpc, Apc, Bpc, Cpc}, {xi, eta, zeta}]
BestFitY = FindFit[PlateConstantsDataY, Hpc + Epc xi + Fpc eta + Gpc zeta,
  {Hpc, Epc, Fpc, Gpc}, {xi, eta, zeta}]
```

Out[601... {Dpc → -1.45501, Apc → 1464.48, Bpc → 29.5276, Cpc → 77.756}
 {Hpc → -0.0655413, Epc → -34.6099, Fpc → 1458.63, Gpc → -181.767}

```
In [605... BestFitXnlm = NonlinearModelFit[PlateConstantsDataX,
  Dpc + Apc xi + Bpc eta + Cpc zeta, {Dpc, Apc, Bpc, Cpc}, {xi, eta, zeta}];
BestFitYnlm = NonlinearModelFit[PlateConstantsDataY,
  Hpc + Epc xi + Fpc eta + Gpc zeta, {Hpc, Epc, Fpc, Gpc}, {xi, eta, zeta}];

BestFitXnlm["ParameterConfidenceIntervalTable"]
BestFitYnlm["ParameterConfidenceIntervalTable"]
```

	Estimate	Standard Error	Confidence Interval
Dpc	-1.45501	6.35922	{-14.0437, 11.1337}
Apc	1464.48	6.69441	{1451.23, 1477.74}
Bpc	29.5276	2.87025	{23.8456, 35.2095}
Cpc	77.756	7.24506	{63.4137, 92.0983}
	Estimate	Standard Error	Confidence Interval
Hpc	-0.0655413	8.52164	{-16.935, 16.8039}
Epc	-34.6099	8.97081	{-52.3685, -16.8512}
Fpc	1458.63	3.84626	{1451.02, 1466.24}
Gpc	-181.767	9.70871	{-200.986, -162.548}

— Pass3 Geometric quantities from plate constants

```
In [613... XCenter = BestFitX[[1, 2]]
YCenter = BestFitY[[1, 2]]
RadiusX = Sqrt[(BestFitX[[2,2]])^2 + (BestFitX[[3,2]])^2 + (BestFitX[[4,2]])^2]
RadiusY = Sqrt[(BestFitY[[2,2]])^2 + (BestFitY[[3,2]])^2 + (BestFitY[[4,2]])^2]
AvePixRadius = (RadiusX + RadiusY) / 2
AvePhysicalRadius = (1736.0 + 1738.1) / 2
AvePixelScale = AvePhysicalRadius / AvePixRadius
ThetaRotObs = ArcTan[-BestFitX[[3,2]] / BestFitY[[3,2]]] (1/Degree)
```

Out[613... -1.45501
 -0.0655413
 1466.84
 1470.32
 1468.58
 1737.05
 1.18281
 -1.1597

— Pass3 Optical libration angles

```
In [625... AEUnRot = RotationMatrix[-ThetaRotObs Degree].{BestFitX[[2,2]]/RadiusX,
  BestFitY[[2,2]]/RadiusY}

AEUnRot[[1]]
ArcCos[AEUnRot[[1]]] (1/Degree)

CGUnRot = RotationMatrix[-ThetaRotObs Degree].{BestFitX[[4,2]]/RadiusX,
```

```

BestFitY[[4,2]]/RadiusY}
LambdaLibObs = -ArcSin[CGUnRot[[1]]] (1/Degree)
BetaLibObs   = ArcSin[-CGUnRot[[2]] / Cos[-ArcSin[CGUnRot[[1]]]]] (1/Degree)

```

```

Out[625... {0.998663, -0.00332754}
0.998663
2.96311
{0.0555002, -0.122526}
-3.18156
7.04883

```

```

In [632... Print["Pass 3 (Corrected) - Libration Results"];
Print["  λ_lib,obs = ", N[LambdaLibObs, 7], " deg"];
Print["  β_lib,obs = ", N[BetaLibObs, 7], " deg"];
Print["  θ_rot      = ", N[ThetaRotObs, 7], " deg"];

```

```

Pass 3 (Corrected) -- Libration Results
\[Lambda]_lib,obs = -3.18156 deg
\[Beta]_lib,obs = 7.04883 deg
\[Theta]_rot      = -1.1597 deg

```

```

In [636... (* Preserve Pass 3 (Corrected) results under unique variable names *)
LambdaLibObsResultPass3 = LambdaLibObs;
BetaLibObsResultPass3   = BetaLibObs;
ThetaRotObsResultPass3  = ThetaRotObs;
RadiusXResultPass3      = RadiusX;
RadiusYResultPass3      = RadiusY;
AvePixScaleResultPass3  = AvePixelScale;
RMSXResultPass3         = Sqrt[Mean[BestFitXnlm["FitResiduals"]^2]];
RMSYResultPass3         = Sqrt[Mean[BestFitYnlm["FitResiduals"]^2]];
Print["Results stored as *ResultPass3"];

```

Results stored as *ResultPass3

Three-Pass Comparison Table

Comparing libration results across the three processing stages alongside the VMA reference.

```

In [650... (* Three-pass comparison; runs after all three passes are complete *)
Print["====="];
Print["      THREE-PASS LIBRATION COMPARISON"];
Print["====="];
Print[StringForm["  `1`  λ = `2` deg  β = `3` deg",
  "Pass 1 (Raw)      : ", N[LambdaLibObsResultRaw, 6], N[BetaLibObsResultRaw,
Print[StringForm["  `1`  λ = `2` deg  β = `3` deg",
  "Pass 2 (Centered) : ", N[LambdaLibObsResultPass2, 6], N[BetaLibObsResultPass2
Print[StringForm["  `1`  λ = `2` deg  β = `3` deg",
  "Pass 3 (Corrected): ", N[LambdaLibObsResultPass3, 6], N[BetaLibObsResultPass3
Print[StringForm["  `1`  λ = `2` deg  β = `3` deg",
  "VMA Reference      : ", N[LambdaLibVMA, 6], N[BetaLibVMA, 6]]];
Print["-----"];
Print[StringForm[" Δλ (Pass3 - VMA) = `1` deg  Δβ (Pass3 - VMA) = `2` deg",
  N[LambdaLibObsResultPass3 - LambdaLibVMA, 4],
  N[BetaLibObsResultPass3 - BetaLibVMA, 4]]];

```



```
=====
THREE-PASS LIBRATION COMPARISON
=====
Pass 1 (Raw)      :  \[Lambda] = -3.09042 deg  \[Beta] = 7.06883 deg
Pass 2 (Centered) :  \[Lambda] = -3.09042 deg  \[Beta] = 7.06883 deg
Pass 3 (Corrected):  \[Lambda] = -3.18156 deg  \[Beta] = 7.04883 deg
VMA Reference     :  \[Lambda] = -2.70000 deg  \[Beta] = 7.50000 deg
-----
\[CapitalDelta]\[Lambda] (Pass3 - VMA) = -0.481562

> deg  \[CapitalDelta]\[Beta] (Pass3 - VMA) = -0.451174 deg
```

Final Comparison: Measured vs VMA Libration

Once the plate solution is complete, the cell below prints your photogrammetrically determined libration angles and compares them to the reference values from VMA.

```
In [668... (* Compare plate-solution libration to VMA reference *)
Print["=== LIBRATION COMPARISON ==="];
Print["Libration in longitude:"];
Print[" Plate solution : ", N[LambdaLibObs], " deg"];
Print[" VMA reference   : ", N[LambdaLibVMA], " deg"];
Print[" Difference      : ", N[LambdaLibObs - LambdaLibVMA], " deg"];
Print[];
Print["Libration in latitude:"];
Print[" Plate solution : ", N[BetaLibObs], " deg"];
Print[" VMA reference   : ", N[BetaLibVMA], " deg"];
Print[" Difference      : ", N[BetaLibObs - BetaLibVMA], " deg"];
Print[];
Print["Position angle:"];
Print[" Plate solution : ", N[ThetaRotObs], " deg"];
Print[" VMA reference   : ", PositionAngleVMA, " deg"];
Print[" Difference      : ", N[ThetaRotObs - PositionAngleVMA], " deg"];
```

```
=== LIBRATION COMPARISON ===
Libration in longitude:
  Plate solution : -3.18156 deg
  VMA reference   : -2.7 deg
  Difference      : -0.481562 deg

Libration in latitude:
  Plate solution : 7.04883 deg
  VMA reference   : 7.5 deg
  Difference      : -0.451174 deg

Position angle:
  Plate solution : -1.1597 deg
  VMA reference   : -9.1 deg
  Difference      : 7.9403 deg
```

Plate Calibration Residual Analysis

A good plate solution requires that the fit residuals are small and randomly distributed. The cells below plot the residuals for both the X - and Y -fits and compute their root-mean-square (RMS) value in pixels, which quantifies the goodness of the plate calibration.

In [706...

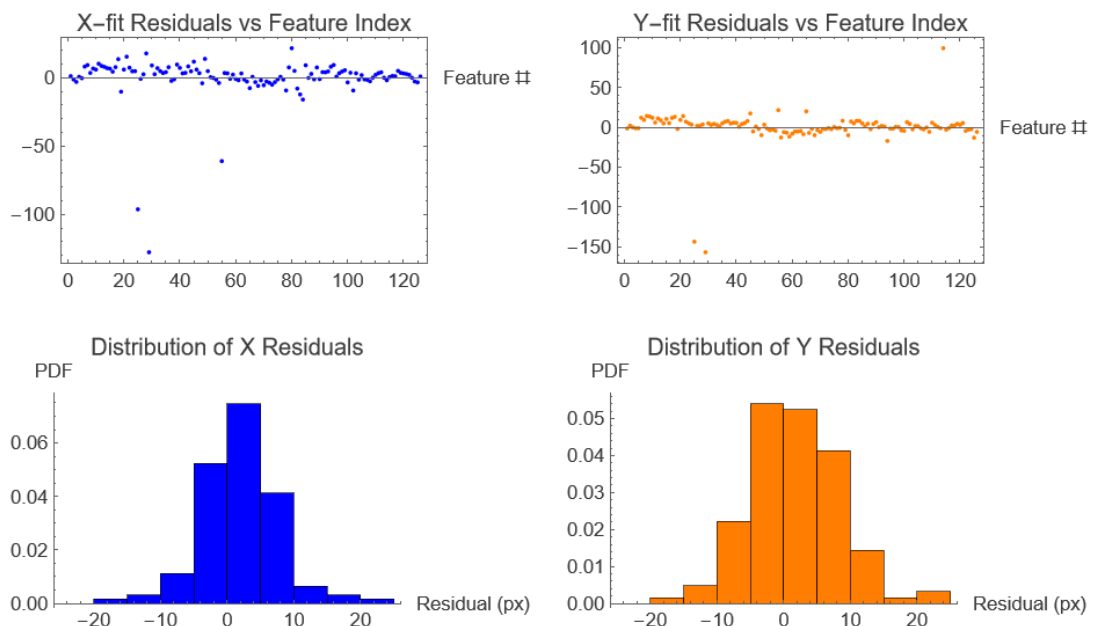
```
ResidualsX = BestFitXnlm["FitResiduals"];
ResidualsY = BestFitYnlm["FitResiduals"];
RMSX = Sqrt[Mean[ResidualsX^2]];
RMSY = Sqrt[Mean[ResidualsY^2]];
Print["RMS residual X-fit : ", RMSX, " px (", RMSX * AvePixelScale, " km)"];
Print["RMS residual Y-fit : ", RMSY, " px (", RMSY * AvePixelScale, " km)"];

featureIndices = Range[FeatureMax];
pResX = ListPlot[Transpose[{featureIndices, ResidualsX}],
  PlotStyle -> Directive[Blue, PointSize[0.012]], PlotLabel -> "X-fit Residuals",
  AxesLabel -> {"Feature #", "Residual (px)"}, PlotRange -> All, Frame -> True,
  GridLines -> {None, {0}}, GridLinesStyle -> {Red, Dashed}];
pResY = ListPlot[Transpose[{featureIndices, ResidualsY}],
  PlotStyle -> Directive[Orange, PointSize[0.012]], PlotLabel -> "Y-fit Residual",
  AxesLabel -> {"Feature #", "Residual (px)"}, PlotRange -> All, Frame -> True,
  GridLines -> {None, {0}}, GridLinesStyle -> {Red, Dashed}];
hResX = Histogram[ResidualsX, Automatic, "PDF",
  PlotLabel -> "Distribution of X Residuals", AxesLabel -> {"Residual (px)", "PDF"}];
hResY = Histogram[ResidualsY, Automatic, "PDF",
  PlotLabel -> "Distribution of Y Residuals", AxesLabel -> {"Residual (px)", "PDF"}];
BonusBResidualGrid = GraphicsGrid[{pResX, pResY}, {hResX, hResY}], ImageSize ->
BonusBResidualGrid
```

RMS residual X-fit : 16.4111 px (19.4111 km)

RMS residual Y-fit : 21.9916 px (26.0118 km)

Out[706...



In [720...

```
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-r",
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-r"]
```

```

Out[720... D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-residual-analysis\

> .pdf

D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
ct-1-residual-analysis\

> .png

```

Selenographic Coordinate Recovery Check

To assess the quality of the plate solution, we can *recover* the selenographic coordinates of each feature from the fitted plate constants and compare them to the catalog values taken from the Virtual Moon Atlas. The residuals in angular coordinates provide an independent quality metric.

```

In [727... (* Recover ( $\lambda$ ,  $\beta$ ) from fitted plate constants for each feature *)
Apc = BestFitX[[2, 2]]; Bpc = BestFitX[[3, 2]]; Cpc = BestFitX[[4, 2]]; Dpc = Be
Epc = BestFitY[[2, 2]]; Fpc = BestFitY[[3, 2]]; Gpc = BestFitY[[4, 2]]; Hpc = Be

RecoveredXi = Table[(PixelCoordinatesX[[i]] - Dpc) / RadiusX, {i, 1, FeatureMax}
RecoveredEta = Table[(PixelCoordinatesY[[i]] - Hpc) / RadiusY, {i, 1, FeatureMax}

RecoveredLambda = Table[
  ArcSin[RecoveredXi[[i]] /
    Sqrt[1 - RecoveredEta[[i]]^2]] (1/Degree), {i, 1, FeatureMax}];
RecoveredBeta = Table[
  ArcSin[RecoveredEta[[i]]] (1/Degree), {i, 1, FeatureMax}];

DeltaLambda = RecoveredLambda - FeatureCoordinatesLong;
DeltaBeta = RecoveredBeta - FeatureCoordinatesLat;

Print["Mean  $|\Delta\lambda|$  = ", Mean[Abs[DeltaLambda]], " deg"];
Print["Mean  $|\Delta\beta|$  = ", Mean[Abs[DeltaBeta]], " deg"];
Print["Max  $|\Delta\lambda|$  = ", Max[Abs[DeltaLambda]], " deg (feature #",
  Position[Abs[DeltaLambda], Max[Abs[DeltaLambda]]][[1,1]], ")"];
Print["Max  $|\Delta\beta|$  = ", Max[Abs[DeltaBeta]], " deg (feature #",
  Position[Abs[DeltaBeta], Max[Abs[DeltaBeta]]][[1,1]], ")"];

(* Scatter: VMA coords vs. recovered coords *)
pLambda = ListPlot[
  {Transpose[{FeatureCoordinatesLong, FeatureCoordinatesLong}],
    Transpose[{FeatureCoordinatesLong, RecoveredLambda}]},
  PlotStyle -> {{Dashed, Gray}, {Blue, PointSize[0.01]}},
  PlotLabel -> "Longitude: Catalog vs Recovered",
  AxesLabel -> {" $\lambda_{VMA}$  (deg)", " $\lambda_{recovered}$  (deg)"},
  PlotLegends -> {"1:1 line", "Recovered"},
  Frame -> True]

pBeta = ListPlot[
  {Transpose[{FeatureCoordinatesLat, FeatureCoordinatesLat}],
    Transpose[{FeatureCoordinatesLat, RecoveredBeta}]},
  PlotStyle -> {{Dashed, Gray}, {Orange, PointSize[0.01]}},
  PlotLabel -> "Latitude: Catalog vs Recovered",

```

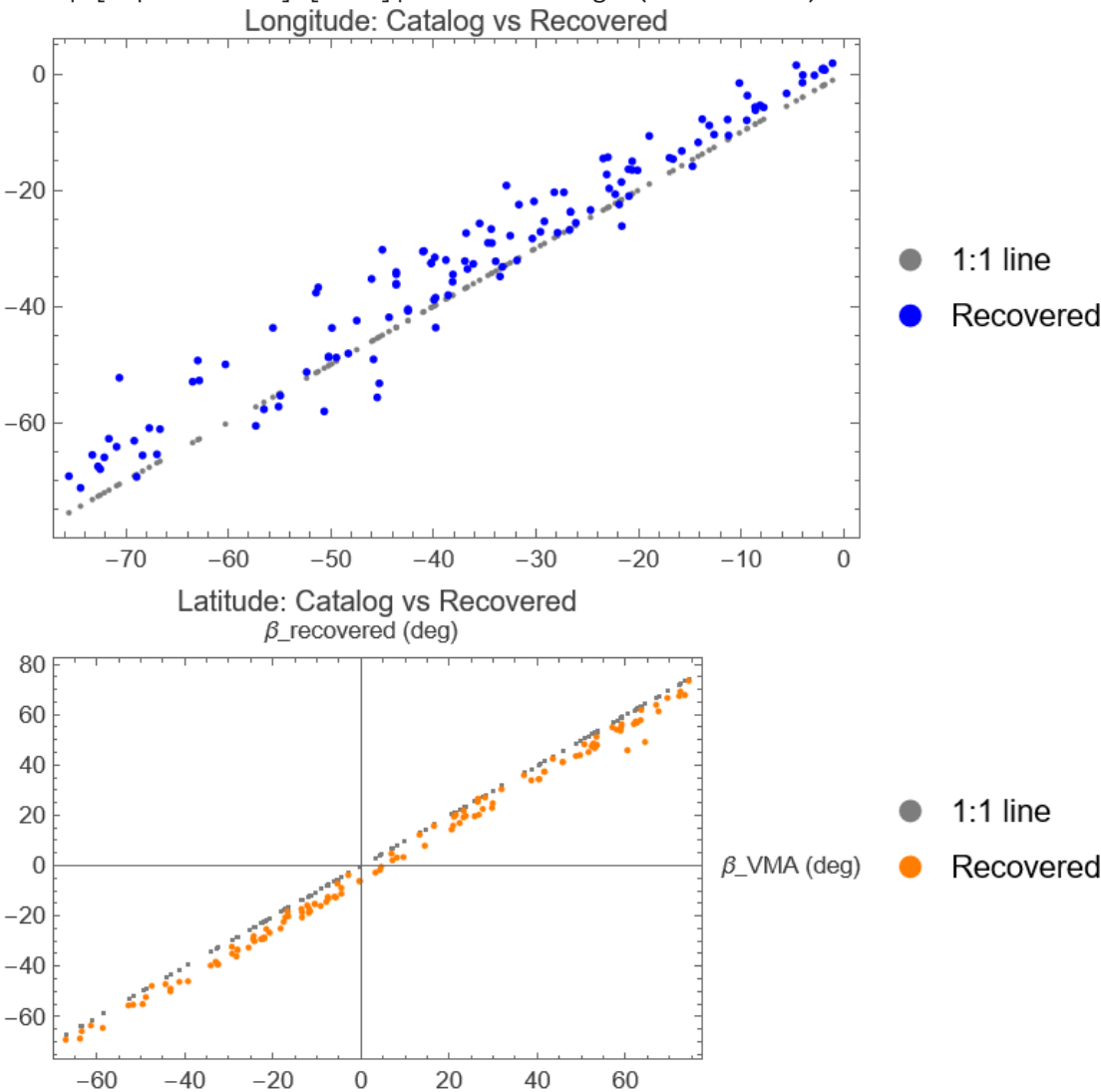
```

AxesLabel -> {"β_VMA (deg)", "β_recovered (deg)"},
PlotLegends -> {"1:1 line", "Recovered"},
Frame -> True]

```

Mean |\\[CapitalDelta]\\[Lambda]| = 4.93315 deg
Mean |\\[CapitalDelta]\\[Beta]| = 4.73661 deg
Max |\\[CapitalDelta]\\[Lambda]| = 18.3295 deg (feature #10)
Max |\\[CapitalDelta]\\[Beta]| = 15.2991 deg (feature #25)

Out[727...



Summary Table of Results

A concise summary of all key numerical results from the project, formatted for easy inclusion in your report.

In [773...

```

Grid[{
  {Style["Quantity", Bold, 12], Style["Value", Bold, 12], Style["Unit", Bold, 12],
    {"Number of features used", FeatureMax, "-"},
    {"Moon distance (FITS header)", rMoon/10^3, "km"},
    {"Mean physical radius (used)", AvePhysicalRadius, "km"},
    {"Angular radius of Moon", N[AveAngularRadiusDeg], "deg"},
    {"Pixel radius (X-axis)", N[RadiusX], "px"},
    {"Pixel radius (Y-axis)", N[RadiusY], "px"},
    {"Average pixel scale", N[AvePixelScale], "km/px"},
    {"Center of Moon X_C", N[XCenter], "px"}
}]

```

```

{"Center of Moon Y_C",      N[YCenter],      "px"},
{"Polar-axis rotation  $\theta$ ", N[ThetaRotObs], "deg"},
{"Libration in longitude  $\lambda_{\text{obs}}$ ", N[LambdaLibObs], "deg"},
{"Libration in latitude  $\beta_{\text{obs}}$ ", N[BetaLibObs], "deg"},
{"RMS residual (X-fit)",    N[RMSX],        "px"},
{"RMS residual (Y-fit)",    N[RMSY],        "px"}
},
Frame -> All,
FrameStyle -> LightGray,
Alignment -> {{Left, Right, Center}},
Background -> {{}}, {GrayLevel[0.85], {White, GrayLevel[0.95]}}},
Spacings -> {2, 1}]

```

Out[773...

Quantity	Value	Unit
Number of features used	126	–
Moon distance (FITS header)	368 567	km
Mean physical radius (used)	1737.05	km
Angular radius of Moon	0.270035	deg
Pixel radius (X-axis)	1466.84	px
Pixel radius (Y-axis)	1470.32	px
Average pixel scale	1.18281	km/px
Center of Moon X_C	–1.45501	px
Center of Moon Y_C	–0.0655413	px
Polar-axis rotation θ	–1.1597	deg
Libration in longitude λ_{obs}	–3.18156	deg
Libration in latitude β_{obs}	7.04883	deg
RMS residual (X-fit)	16.4111	px
RMS residual (Y-fit)	21.9916	px

In [110...

```

(* First capture the table as a graphic, then export *)
SummaryTableGraphic = Rasterize[
  Grid[{
    {Style["Quantity", Bold, 12], Style["Value", Bold, 12], Style["Unit", Bold,
    {"Number of features used",      FeatureMax,      "-"},
    {"Moon distance (FITS header)",   rMoon/10^3,     "km"},
    {"Mean physical radius (used)",    AvePhysicalRadius, "km"},
    {"Angular radius of Moon",        N[AveAngularRadiusDeg], "deg"},
    {"Pixel radius (X-axis)",          N[RadiusX],      "px"},
    {"Pixel radius (Y-axis)",          N[RadiusY],      "px"},
    {"Average pixel scale",            N[AvePixelScale], "km/px"},
    {"Center of Moon X_C",             N[XCenter],      "px"},
    {"Center of Moon Y_C",             N[YCenter],      "px"},
    {"Polar-axis rotation  $\theta$ ", N[ThetaRotObs], "deg"},

```

```

{"λ_obs (Pass 3 / Corrected)", N[LambdaLibObs], "deg"},
{"β_obs (Pass 3 / Corrected)", N[BetaLibObs], "deg"},
{"λ_VMA reference", N[LambdaLibVMA], "deg"},
{"β_VMA reference", N[BetaLibVMA], "deg"},
{"RMS residual X-fit", N[RMSX], "px"},
{"RMS residual Y-fit", N[RMSY], "px"}
},
Frame -> All, FrameStyle -> LightGray,
Alignment -> {{Left, Right, Center}},
Background -> {{}} , {GrayLevel[0.85], {White, GrayLevel[0.95]}}},
Spacings -> {2, 1}
], ImageResolution -> 300];
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-s
Export["D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-project-1-s

```

Out[110... D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
 ct-1-summary-table.pdf
 D:/AE-451/Project1/Figures/kaoutar-ammara-spring-2026-ae-451-proje
 ct-1-summary-table.png

"Introduction to lunar photogrammetry and optical libration determination" by Fabrizio Pinto is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.
<https://creativecommons.org/licenses/by/4.0/>

Attribution — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.